Algoritmos de optimización - Seminario

Nombre y Apellidos: Sergio Monrabal Romero y Alex Sanz Morant
Problema:

> 3. Combinar cifras y operaciones

Descripción del problema:(copiar enunciado)

• El problema consiste en analizar el siguiente problema y diseñar un algoritmo que lo resuelva.

• Disponemos de las 9 cifras del 1 al 9 (excluimos el cero) y de los 4 signos básicos de lasoperaciones fundamentales: suma(+), resta(-), multiplicación(*) y división(/)

• Debemos combinarlos alternativamente sin repetir ninguno de ellos para obtener una cantidad dada. Un ejemplo sería para obtener el 4: 4+2-6/3*1 = 4

• Debe analizarse el problema para encontrar todos los valores enteros posibles planteando las siguientes cuestiones:

- ¿Qué valor máximo y mínimo se pueden obtener según las condiciones del problema?
- ¿Es posible encontrar todos los valores enteros posibles entre dicho mínimo y máximo ?

• Nota: Es posible usar la función de python “eval” para evaluar una expresión

(*) La respuesta es obligatoria

(*)¿Cuantas posibilidades hay sin tener en cuenta las restricciones?

¿Cuantas posibilidades hay teniendo en cuenta todas las restricciones.

Respuesta

En el problema se cuenta con dos restricciones, por un lado no se pueden repetir la cifra y por el otro no se puede repetir la operación. Teniendo en cuenta esto las posiblidades que hay son las siguientes:
-La primera cifra se elegirá entre 9 posibles.
-La primera operación se elegirá entre las 4 posibles.
-La segunda cifra se elegirá entre 8 posibles.
-La segunda operación se elegirá entre las 3 posibles.
-La tercera cifra se elegirá entre 7 posibles.
-La tercera operación se elegirá entre las 2 posibles.
-La cuarta cifra se elegirá entre 6 posibles.
-La cuarta operación se elegirá entre las 1 posibles.

-La quinta cifra se elegirá entre 5 posibles.
Así pues, multiplicando las posibles elecciones de cifras y números anterior, tendremos que la cantidad de operaciones posibles será de 9!=362880.

Sin embargo, si no tenemos en cuenta las restricciones anteriores, no disminuirá el número de elecciones de cifra y operador que realicemos, quedando las elecciones de la siguiente manera:
-La primera cifra se elegirá entre 9 posibles.
-La primera operación se elegirá entre las 4 posibles.
-La segunda cifra se elegirá entre 9 posibles.
-La segunda operación se elegirá entre las 4 posibles.
-La tercera cifra se elegirá entre 9 posibles.
-La tercera operación se elegirá entre las 4 posibles.
-La cuarta cifra se elegirá entre 9 posibles.
-La cuarta operación se elegirá entre las 4 posibles.
-La quinta cifra se elegirá entre 9 posibles.
Por lo tanto el número de posiblidades, sin restricciones, será de $9^5 * 4^4$ = 15116544.

Modelo para el espacio de soluciones
(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Respuesta

La estructura de datos que mejor se adapta al problema es la lista. Las listas son un tipo de estructura de datos muy versátil que cuentan con una serie de funciones built-in que son bastante útiles para la resolución de nuestro problema. En particular, las funciones por las cuales hemos decidido usar este tipo de estructura han sido *pop* e *insert*.

La función *pop* nos ha permitido extraer un elemento de la lista y borrarlo de la misma, gracias a lo cual nos hemos podido asegurar de no repetir números u operaciones.

De forma complementaria, la función *insert* nos ha permitido volver a insertar aquellos elementos que incialmente habíamos quitado de la lista para comprobar una posible solución, la cual hemos descartado.

Según el modelo para el espacio de soluciones
(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Respuesta

El problema que nos ocupa no se trata de ningún problema cuyo objetivo sea la optimización de algún sistema, sino que se trata del análisis y el diseño de un algoritmo que sea capaz de calcular un entero dadas ciertas condiciones. Por este motivo, no

podemos decir que exista ninguna función objetivo ya que estas, por definición, están asociadas a un problema de optimización.

Análogamente podemos concluir que no se trata de ningún problema de maximización ni minimización, ya que en ese caso estaríamos ante un problema de optimización.

Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

In [ ]:
```
#Fuerza Bruta

#Se definen dos listas con los números y las operaciones
lista_operaciones = ["+","-","*","/"]
lista_numeros = [1,2,3,4,5,6,7,8,9]

def operaciones_para(target):
    nums = lista_numeros
    ops = lista_operaciones

    #Iteramos sobre la lista de números tomando 5 números distintos para comprob
    for num_uno in nums:
        for num_dos in nums:
            if num_dos == num_uno:
                continue
            for num_tres in nums:
                if num_tres == num_uno or num_tres == num_dos:
                    continue
                for num_cuatro in nums:
                    if num_cuatro == num_uno or num_cuatro == num_dos or num_cua
                        continue
                    for num_cinco in nums:
                        if (num_cinco == num_uno or num_cinco == num_dos or
                            num_cinco == num_tres or num_cinco == num_cuatro):
                            continue

                        #Iteramos sobre la lista de operaciones tomando 4 distin
                        for op_uno in ops:
                            for op_dos in ops:
                                if op_dos == op_uno:
                                    continue
                                for op_tres in ops:
                                    if op_tres == op_uno or op_tres == op_dos:
                                        continue
                                    for op_cuatro in ops:
                                        if op_cuatro == op_uno or op_cuatro == o
                                            continue

                                        #Evaluamos si la expresión con las opera
                                        expr = f"{num_uno}{op_uno}{num_dos}{op_d
                                        if eval(expr) == target:
                                            return expr
    return None #Si no encontramos ninguna combinación devolvemos None

for s in range(-70, 80):
    print(f"{s}: {operaciones_para(s)}")
```

```
-70: None
-69: 1+4/2-8*9
-68: 1+6/2-8*9
-67: 2/1+3-8*9
-66: 2/1+4-8*9
-65: 2/1+5-8*9
-64: 2/1+6-8*9
-63: 2/1+7-8*9
-62: 3/1+7-8*9
-61: 4/1+7-8*9
-60: 1+4/2-7*9
-59: 1+6/2-7*9
-58: 1-7*9+8/2
-57: 2/1+4-7*9
-56: 2/1+5-7*9
-55: 2/1+6-7*9
-54: 3/1+6-7*9
-53: 1+4/2-7*8
-52: 1+6/2-7*8
-51: 1+4/2-6*9
-50: 2/1+4-7*8
-49: 1-6*9+8/2
-48: 2/1+4-6*9
-47: 2/1+5-6*9
-46: 3/1+5-6*9
-45: 1+4/2-6*8
-44: 1-6*8+9/3
-43: 2/1+3-6*8
-42: 1+4/2-5*9
-41: 1-5*9+6/2
-40: 1-5*9+8/2
-39: 1+4/2-6*7
-38: 1-6*7+9/3
-37: 1+4/2-5*8
-36: 1-5*8+6/2
-35: 2/1+3-5*8
-34: 2/1+4-5*8
-33: 1-4*9+6/3
-32: 1+3-8/2*9
-31: 1+4-8/2*9
-30: 1-5*7+8/2
-29: 1-4*8+6/3
-28: 1-4*8+6/2
-27: 1+4/2-5*6
-26: 1-5*6+9/3
-25: 1-4*7+6/3
-24: 1+3-7/2*8
-23: 1+3-6/2*9
-22: 1-3*9+8/2
-21: 1+2-8/3*9
-20: 1+3-6/2*8
-19: 1+4-6/2*8
-18: 1+2-7/3*9
-17: 1+3-6/2*7
-16: 1+3-5/2*8
-15: 1+2-6/3*9
-14: 1+3-4/2*9
-13: 1+2-6/3*8
-12: 1+2-5/3*9
-11: 1+2-6/3*7
```

-10: 1-2*7+9/3
-9: 1+2-4/3*9
-8: 1-2*6+9/3
-7: 1+2*3/6-9
-6: 1+2*3/6-8
-5: 1+2*3/6-7
-4: 1+2/3*6-9
-3: 1+2-3/4*8
-2: 1+2*3/6-4
-1: 1+2-3/6*8
0: 1+2/3*6-5
1: 1+2-3*4/6
2: 1-2/3*6+5
3: 1-2+3/6*8
4: 1-2*3/6+4
5: 1-2+3/4*8
6: 1-2/3*6+9
7: 1-2*3/6+7
8: 1-2*3/6+8
9: 1-2*3/6+9
10: 1+2*6-9/3
11: 1-2+4/3*9
12: 1+2*7-9/3
13: 1-2+6/3*7
14: 1-2+5/3*9
15: 1-2+6/3*8
16: 1-3+4/2*9
17: 1-2+6/3*9
18: 1-3+5/2*8
19: 1-3+6/2*7
20: 1-2+7/3*9
21: 1-4+6/2*8
22: 1-3+6/2*8
23: 1-2+8/3*9
24: 1+3*9-8/2
25: 1-3+6/2*9
26: 1-3+7/2*8
27: 1+4*7-6/3
28: 1+5*6-9/3
29: 1-4/2+5*6
30: 1+4*8-6/2
31: 1+4*8-6/3
32: 1+5*7-8/2
33: 1-4+8/2*9
34: 1-3+8/2*9
35: 1+4*9-6/3
36: 2/1+5*8-6
37: 3/1-2+4*9
38: 1+5*8-6/2
39: 1-4/2+5*8
40: 1+6*7-9/3
41: 1-4/2+6*7
42: 1+5*9-8/2
43: 1+5*9-6/2
44: 1-4/2+5*9
45: 2/1-5+6*8
46: 1+6*8-9/3
47: 1-4/2+6*8
48: 2/1+6*9-8
49: 2/1+6*9-7

```
50: 3/1+6*9-7
51: 1+6*9-8/2
52: 2/1-4+6*9
53: 1-4/2+6*9
54: 1-6/2+7*8
55: 1-4/2+7*8
56: 2-6/3+7*8
57: 2/1+7*9-8
58: 3/1+7*9-8
59: 2/1-6+7*9
60: 1+7*9-8/2
61: 1-6/2+7*9
62: 1-4/2+7*9
63: 2-6/3+7*9
64: 3/1-2+7*9
65: 4/1-2+7*9
66: 5/1-2+7*9
67: 2/1-7+8*9
68: 2/1-6+8*9
69: 2/1-5+8*9
70: 1-6/2+8*9
71: 1-4/2+8*9
72: 2-6/3+8*9
73: 3/1-2+8*9
74: 4/1-2+8*9
75: 5/1-2+8*9
76: 6/1-2+8*9
77: 7/1-2+8*9
78: None
79: None
```

Calcula la complejidad del algoritmo por fuerza bruta

Respuesta

El algoritmo anterior tiene 9 bucles for anidados, 5 de los cuales que recorren la lista de números (de tamaño N) y los otros 4 recorren la lista de operaciones (de tamaño M). Considerando que el resto de operaciones dentro de la función son despreciables debido a que tienen orden constante y que la cantidad de operaciones que se puede pasar está acotada tenemos que el algoritmo tiene una complejidad de $\boldsymbol{O}(N^5)$.

(*)Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Respuesta

In [ ]:
```python
#Se definen dos listas con los números y las operaciones
lista_operaciones=["+","-","*","/"]
lista_numeros=[1,2,3,4,5,6,7,8,9]

def operaciones_para(target):
  #Se crea una copia de las listas anteriores
  temp_ops = lista_operaciones[:]
  temp_nums = lista_numeros[:]

  def buscarDigito(stage, actualExpr):
```

```
    #Pequeña poda si ya es imposible llegar al valor cuando queda solo "+" y "-"
    if stage > 0:
      solo_mas_menos = True
      for op in temp_ops: #M operaciones
        if op != "+" and op != "-":#Operaciones constantes
          solo_mas_menos = False
          break
      if solo_mas_menos:
        actualVal = eval(actualExpr)
        s = 0
        for x in temp_nums:#N operaciones
          s += x
        if target < actualVal - s or target > actualVal + s:#Operaciones constan
          return None

    #Si estamos realizando el último paso evaluamos la expresión
    if stage == 5: #Operaciones constantes
      if eval(actualExpr) == target:
        return actualExpr
      return None

    #Se aplica Backtracking para obtener los números y las operaciones
    for i in range(len(temp_nums)): #N operaciones
      actual_dig = temp_nums.pop(i)
      if stage == 0:
          ans = buscarDigito(stage + 1, str(actual_dig))
      else:
        ans = None
        for j in range(len(temp_ops)): #M operaciones
          actual_op = temp_ops.pop(j) #Operaciones constantes

          ans = buscarDigito(stage + 1, actualExpr + actual_op + str(actual_dig)

          temp_ops.insert(j, actual_op) #Operaciones constantes
          if ans is not None: #Operaciones constantes
            break

      temp_nums.insert(i, actual_dig) #Operaciones constantes
      if ans is not None: #Operaciones constantes
        return ans

    return None

  return buscarDigito(0, "")

for s in range(20):
  print(str(s)+": ",operaciones_para(s))
```

```
0:  1+2/3*6-5
1:  1+2-3*4/6
2:  1+2/3*9-5
3:  1+2/3*9-4
4:  1-2*3/6+4
5:  1+2*3-8/4
6:  1+2*4-9/3
7:  1+2*4-6/3
8:  1+2*5-9/3
9:  1+2*5-6/3
10:  1+2*6-9/3
11:  1+2*6-8/4
12:  1+2*7-9/3
13:  1+2*7-6/3
14:  1+2*8-9/3
15:  1+2*8-6/3
16:  1-3+4/2*9
17:  1+2*9-6/3
18:  1+3*7-8/2
19:  1+3*7-6/2
```

(*)Calcula la complejidad del algoritmo

Respuesta

En este caso el algoritmo anterior tiene un orden de complejidad $\mathcal{O}(N^6)$ en el peor de los casos (ya que la poda aumenta la complejidad de $\mathcal{O}(N^5)$ a $\mathcal{O}(N^6)$). Sin embargo, como la mitad de las combinaciones de operaciones acaban en suma o resta y una cuarta parte acaba en suma y resta, en promedio, este algoritmo tendrá orden de comlejidad $\mathcal{O}(N^5)$ y realizará menos operaciones que el algoritmo de fuerza bruta.

- ¿Qué valor máximo y mínimo se pueden obtener según las condiciones del problema?

Podemos establecer unas cotas superior e inferior a la operación y utilizar el algoritmo que hemos diseñado para encontrar el máximo y el mínimo.

Trivialmente, el número *99+9-1/1=90 puede ejercer de cota superior, mientras que 1/1+1-99=-79* puede ejercer de cota inferior.

Aplicamos el algoritmo:

In [ ]:
```
for i in range(90,0,-1):
  x = operaciones_para(i)
  if x is not None:
    print(x, "=", eval(x))
    break
```

```
7/1-2+8*9 = 77.0
```

In [ ]:
```
for i in range(-79,0):
  x = operaciones_para(i)
  if x is not None:
    print(x, "=", eval(x))
    break
```

```
1+4/2-8*9 = -69.0
```

Por tanto, el máximo es 77 y el mínimo es -69

- ¿Es posible encontrar todos los valores enteros posibles entre dicho mínimo y máximo ?

Ejecutando el siguiente código, podemos comprobar como el algoritmo es capaz de encontrar todos los números entre el -69 y el 77:

In [ ]:
```
comprobacion = []
for s in range(-68,76):
  comprobacion.append(eval(operaciones_para(s)))
if list(range(-68,76)) == comprobacion:
  print("Es capaz.")
else:
  print("No es capaz.")
```

```
Es capaz.
```

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Respuesta

In [ ]:
```
for s in range(20):
  n = random.randint(-70, 80)
  print(str(n)+": ",operaciones_para(n))
```

Aplica el algoritmo al juego de datos generado

Respuesta

In [ ]:
```
40:  1+6*7-9/3
26:  1+3*9-4/2
14:  1+2*8-9/3
26:  1+3*9-4/2
45:  2/1-5+6*8
69:  2/1-5+8*9
-54:  3/1+6-7*9
-8:  1-2*6+9/3
27:  1+4*7-6/3
33:  1+4*9-8/2
25:  1+3*9-6/2
-22:  1-3*9+8/2
-3:  1+2-3/4*8
-62:  3/1+7-8*9
69:  2/1-5+8*9
-7:  1+2*3/6-9
-55:  2/1+6-7*9
-65:  2/1+5-8*9
-26:  1-5*6+9/3
-26:  1-5*6+9/3
```

Describe brevemente las lineas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Un posible avance en el problema podría ser encontrar más cotas que pudiesen mejorar el algoritmo.